

Reference Architecture Monitoring

This directory contains configuration to provide observability into other GitLab instances (not GitLab.com).

This is based off the same Service-Level Monitoring and Saturation Monitoring metrics used to monitor GitLab.com.

Each sub-directory contains a specific reference architecture, although for now, there is only one:

1. **get-hybrid/**: this provides monitoring configuration, dashboards and alerts for a GitLab Environment Toolkit (GET) Hybrid Kubernetes environment.

Warning about Completeness

This is, at present, a work-in-progress. The plan is to start with a small subset of required metrics and expand it until the configuration covers all metrics critical to the operation of a GitLab instance.

The epic tracking this effort is here: <https://gitlab.com/groups/gitlab-com/-/epics/1721>. For up-to-date progress on the effort, consult the epic.

How to use

From your chosen reference-architecture sub-directory, deploy:

- **config/prometheus-rules/*.yml** into your Prometheus
 - Multiple files exist here to prevent scenarios where the Prometheus Operator is unable to create ConfigMaps if files become too large in size.
- **config/dashboards/*.json** to Grafana

Details of how to deploy will vary by your chosen configuration management tooling which is beyond the scope of this documentation.

Ensure you are scraping the metrics from all required sub-systems, using the correct job name. Precise details of what and how to scrape will vary by reference architecture, so the list below is to provide general guidance and commentary without being prescriptive, although it does assume you are using consul with **monitoring_service_discovery** enabled (see <https://docs.gitlab.com/ee/administration/monitoring/prometheus/>). You may need to refer to the service definitions (**src/services/*.jsonnet**) to clarify some details.

Service	Scrape details	Scrape job name required	Notes
consul	-	-	Looks for pods in the 'consul' namespace; monitoring is kubernetes-level only
gitaly	-	gitaly	gitaly consul service
gitlab-shell	-	-	Re-uses praefect metrics. This is a weak proxy until gitlab-shell has more accessible metrics (see runbooks#88)
praefect	-	praefect	praefect consul service
redis	We provide client-side metrics emitted by any rails endpoint, webservice/sidekiq	-	-
registry	In kubernetes, the 'registry-prometheus' port	scrape job must be named praefect	-
sidekiq	-	/metrics	-

Service	Scrape details	Scrape job name required	Notes
webservice (rails)	/-/metrics ; in kubernetes, on the http-webservice port	gitlab-rails	-
webservice (workhorse)	/metrics ; in kubernetes, on the http-workhorse-exporter port	gitlab-rails	-

Generating a Customized Set of Recording Rules, Alerts, and Dashboards

Generation Steps

It's possible to customize the configuration of the reference architecture to suit your GitLab deployment.

- Step 1: Clone this repository locally: `git clone git@gitlab.com:gitlab-com/runbooks.git`
- Step 2: Check which version of `jsonnet-tool` is required by consulting the `.tool-versions` file, and install it from <https://gitlab.com/gitlab-com/gl-infra/jsonnet-tool/-/releases>. Alternatively, follow the **Contributor Onboarding** steps in `README.md` to setup your local development environment. This approach will use `asdf` to install the correct version of `jsonnet-tool` automatically.
- Step 3: create a directory which will contain your local overrides. `mkdir overrides`.
- Step 4: in the `overrides` directory, create an `gitlab-metrics-options.libsonnet` file containing the configuration options. Documentation around possible options is available in the Options section later in the documentation. Reviewing the default options can shed light on configuration options available.
- Step 5: optionally add local services in the `overrides` directory you want to include. Review some services defined in `/reference-architectures/get-hybrid/src/services` to understand how such a service would look like.

```
// overrides/gitlab-metrics-options.libsonnet
{
  // Disable praefect
  praefect: {
    enable: false,
  }
  // Add your own locally defined services
  services: [
```

```

import 'logging.libsonnet',
],
}

```

- Step 6: create a directory which will contain your custom recording rules and Grafana dashboards: `mkdir output`.
- Step 7: use the `generate-reference-architecture-config.sh` script to generate your custom configuration.

```

# generate a custom configuration, using the `get-hybrid` reference architecture,
# emitting configuration to the `output` directory, and reading overrides from the
# `overrides/` directory.

```

```

runbooks/scripts/generate-reference-architecture-config.sh \
  runbooks/reference-architectures/get-hybrid/src/ \
  output/ \
  overrides/

```

- Step 7: install the recording rules from `output/prometheus-rules/*.yaml` into your Prometheus configuration. Depending on the deployment, this can be done with the Kube Prometheus Stack helm chart, local file deployment or another means.
- Step 8: install the Grafana dashboards from `output/dashboards/*.json` into your Grafana instance. The means of deployment will depend on the local configuration.

Options

The following configuration options are available in `gitlab-metrics-options.libsonnet`.

Option	Type	Default	Description
<code>elasticacheMonitoring</code>	Boolean	<code>false</code>	Set to <code>true</code> to enable AWS Elasticache monitoring.
<code>minimumSamplesForMonitoring</code>	Integer	3600	Minimum operation rate thresholds. This is to avoid low-volume, noisy alerts. See service-level-monitoring.md for more details.

Option	Type	Default	Description
<code>minimumOpsRateForMonitoring</code>		null	Minimum operation rate thresholds. This is to avoid low-volume, noisy alerts. See service-level-monitoring.md for more details.
<code>praefect.enable</code>	Boolean	true	Set to false to disable Praefect monitoring. This is usually done when Praefect/Gitaly Cluster is disabled in GitLab Environment Toolkit with <code>praefect_node_count = 0</code>
<code>services</code>	Array	empty	Import any customized service monitoring. For examples see reference-architectures/get-hybrid/src/services
<code>saturationMonitoring</code>	Array	empty	Import any customized saturation monitoring. For examples see reference-architectures/get-hybrid/src/services
<code>rdsInstanceRAMGB</code>	int	null	Configure the size of the RAM for a given RDS instance. Specified in unit GB.

Option	Type	Default	Description
<code>rdsMaxStorageAllocationGB</code>	int	null	Configure the size of the maximum allocated storage for a given RDS instance. Specified in unit GB.
<code>rdsMaxConnections</code>	int	null	Configure the count of the maximum allowed connections for a given RDS instance. Specified as count.
<code>rdsMonitoring</code>	Boolean	false	Set to <code>true</code> to enable AWS RDS monitoring.
<code>useGitlabSSHD</code>	Boolean	false	Set to <code>true</code> to use GitLab SSHD instead of GitLab Shell.
<code>toolingLinks.opensearchHostname</code>	String	null	If provided, is the hostname of an OpenSearch instance with logs for the instance, and enables the generation of links to these logs from Grafana dashboards
<code>toolingLinks.defaultIndexPattern</code>	String	null	Mandatory if <code>opensearchHostname</code> is provided; the default index pattern where logs can be found in OpenSearch

Option	Type	Default	Description
<code>toolingLinks.indexPatterns</code>	Object	<code>{}</code>	Optional; is a map from index catalog entry names to OpenSearch index pattern names, to override the defaultIndexPattern if needed
<code>apdexThresholds.gitlabShell.satisfied</code>	float	<code>3.0</code>	Optional; sets the satisfied apdex threshold for the <code>gitlab-shell</code> service. Must be one of the defined prometheus buckets.
<code>apdexThresholds.gitlabShell.tolerated</code>	float	<code>6.0</code>	Optional; sets the tolerated apdex threshold for the <code>gitlab-shell</code> service. Must be one of the defined prometheus buckets.
<code>apdexThresholds.gitlab.satisfied</code>	float	<code>0.5</code>	Optional; sets the satisfied apdex threshold for the <code>gitlab</code> service. Must be one of the defined prometheus buckets.
<code>apdexThresholds.gitlab.tolerated</code>	float	<code>1.0</code>	Optional; sets the tolerated apdex threshold for the <code>gitlab</code> service. Must be one of the defined prometheus buckets.

Option	Type	Default	Description
<code>apdexThresholds.praefect.satisfied</code>	<code>float</code>	<code>0.5</code>	Optional; sets the satisfied apdex threshold for the praefect service. Must be one of the defined prometheus buckets.
<code>apdexThresholds.praefect.tolerated</code>	<code>float</code>	<code>1.0</code>	Optional; sets the tolerated apdex threshold for the praefect service. Must be one of the defined prometheus buckets.
<code>monitoring.sidekiq</code>	<code>Object</code>	<code>{}</code>	Optional. Empty <code>{}</code> means to use default thresholds for all SLIs. Otherwise, this is a map from SLI name to an object of shard name to a map of threshold overrides. Shards specified in this will be excluded from the default SLO and have a distinct SLO created for them with the override provided. Example:

Option	Type	Default	Description
<code>monitoring.gitaly</code>	Object	<code>{}</code>	Optional. Empty <code>{}</code> means to use default thresholds. Otherwise, specify <code>monitoringThresholds</code> with <code>apdexScore</code> and/or <code>errorRatio</code> overrides. Example:
<code>monitoring.webserver</code>	Object	<code>{}</code>	Optional. Empty <code>{}</code> means to use default thresholds. Otherwise, specify <code>monitoringThresholds</code> with <code>apdexScore</code> and/or <code>errorRatio</code> overrides. Example:
<code>monitoring.nginx</code>	Object	<code>{}</code>	Optional. Empty <code>{}</code> means to use default thresholds. Otherwise, specify <code>monitoringThresholds</code> with <code>apdexScore</code> and/or <code>errorRatio</code> overrides. Example:

Option	Type	Default	Description
monitoring.nginx.alertWindows	Array	['1h', '6h', '3d']	Configures which MWMBR alert windows are used for the nginx service. Valid values are 1h, 6h, 3d as defined in libsonnet-net/mwmbr/multiburn_factors.libsonnet.